

TP 6 : Apprentissage supervisé et classification naïve bayésienne

Compléments de Mathématiques 2 - L3 MIASHS

Lola Etiévant

2026-03-02

Table of contents

1	Préparation des données	2
1.1	Chargement du jeu de données	2
1.2	Vérification des types	2
2	Classifieur bayésien naïf	3
2.1	Rappel	3
2.2	Variables explicatives quantitatives	3
2.3	Variables explicatives utilisées pour la prédiction	5
3	Classifieur naïf bayésien avec Holdout	6
3.1	Partage du jeu de données	6
3.2	Gestion des données manquantes	6
3.3	Phase d'apprentissage	6
3.4	Prédiction des probabilités	10
3.5	Prédiction et évaluation	10
3.6	Pour aller plus loin : importance des variables	12
4	Classifieur naïf bayésien avec 10-fold cross-validation	13
4.1	Assignation des folds	13
4.2	Boucle de validation croisée	13
4.3	Résultats par fold	14
4.4	Moyennes cross-validées	15
5	Pour aller plus loin	15
5.1	Bonus : version catégorielle de Age	15
5.1.1	Avec Holdout	16

5.1.2 Avec 10-fold CV	17
5.2 Bonus : version catégorielle de Fare	18

1 Préparation des données

1.1 Chargement du jeu de données

```

qualitative_vars <- read.csv("../TP2/titanic_pre_processed_qualitative_vars.csv", stringsAsFactors=FALSE)
quantitative_vars <- read.csv("../TP2/titanic_pre_processed_quantitative_vars.csv")
target <- read.csv("../TP2/titanic_pre_processed_target.csv")
target <- as.factor(target$x)

```

Suppression des observations avec valeurs manquantes pour Sex (seulement 2 sur 876, soit < 0.2%) :

```

isMissingSex <- which(is.na(qualitative_vars$Sex) | qualitative_vars$Sex == "")
if (length(isMissingSex) > 0) {
  qualitative_vars <- qualitative_vars[-isMissingSex, ]
  quantitative_vars <- quantitative_vars[-isMissingSex, ]
  target <- target[-isMissingSex]
}

```

1.2 Vérification des types

```

qualitative_vars$Pclass <- as.factor(qualitative_vars$Pclass)
qualitative_vars$Sex <- as.factor(qualitative_vars$Sex)
qualitative_vars$withFam <- as.factor(qualitative_vars$withFam)

str(qualitative_vars)

```

```

'data.frame': 874 obs. of 4 variables:
 $ Age_cat: Factor w/ 4 levels "Adolescent","Adulte",...: 2 2 2 2 2 NA 2 3 2 1 ...
 $ Pclass : Factor w/ 3 levels "1","2","3": 3 1 3 1 3 3 1 3 3 2 ...
 $ Sex : Factor w/ 3 levels "", "female", "male": 3 2 2 2 3 3 3 3 2 2 ...
 $ withFam: Factor w/ 2 levels "0","1": 2 2 1 2 1 1 1 2 2 2 ...

```

```
str(quantitative_vars)
```

```
'data.frame':  874 obs. of  5 variables:
 $ Age  : num  22 38 26 35 35 NA 54 2 27 14 ...
 $ SibSp: int   1 1 0 1 0 0 0 3 0 1 ...
 $ Parch: int   0 0 0 0 0 0 0 1 2 0 ...
 $ Fam  : int   1 1 0 1 0 0 0 4 2 1 ...
 $ Fare : num   7.25 71.28 7.92 53.1 8.05 ...
```

```
str(target)
```

```
Factor w/ 2 levels "Décédé","Survivant": 1 2 2 2 1 1 1 1 2 2 ...
```

2 Classifieur bayésien naïf

2.1 Rappel

Hyperparamètre

Il n'y a **pas d'hyperparamètre** à choisir avec la méthode de classification naïve bayésienne. Il n'y a donc pas besoin de jeu de validation : on partage le jeu de données en **deux parties** seulement (Train / Test).

2.2 Variables explicatives quantitatives

Encoding et normalisation

Avec le classifieur naïf bayésien, on n'a **pas besoin** de transformer les variables qualitatives (dummy coding ou label encoding) ni de normaliser/standardiser les variables quantitatives. Le classifieur travaille directement avec les distributions conditionnelles de chaque variable. En revanche, on fait l'hypothèse que les variables explicatives sont **indépendantes conditionnellement à la target**, et pour les variables quantitatives on suppose qu'elles suivent une **loi normale** conditionnellement à la target.

On exclut Fam, Parch et SibSp puisqu'on utilisera `withFam`. Vérifions l'hypothèse de normalité pour Age et Fare :

```
hist(quantitative_vars$Age, breaks = 30, col = "steelblue",
     main = "Histogramme de Age", xlab = "Age")
```

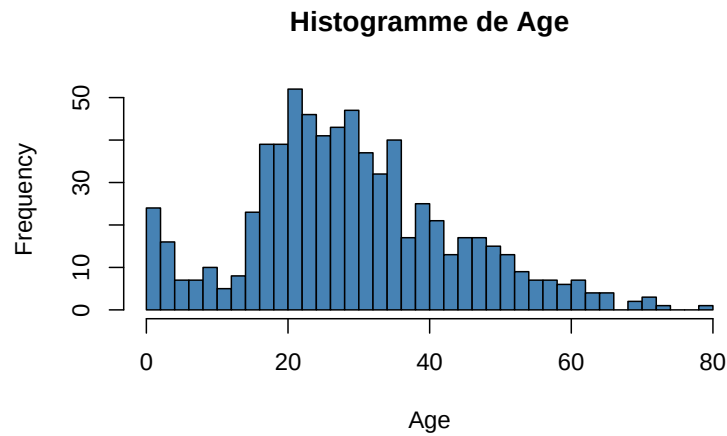


Figure 1: Distribution de Age

Age semble à peu près suivre une distribution unimodale, compatible avec l'hypothèse gaussienne.

```
hist(quantitative_vars$Fare, breaks = 30, col = "coral",
     main = "Histogramme de Fare", xlab = "Fare")
```

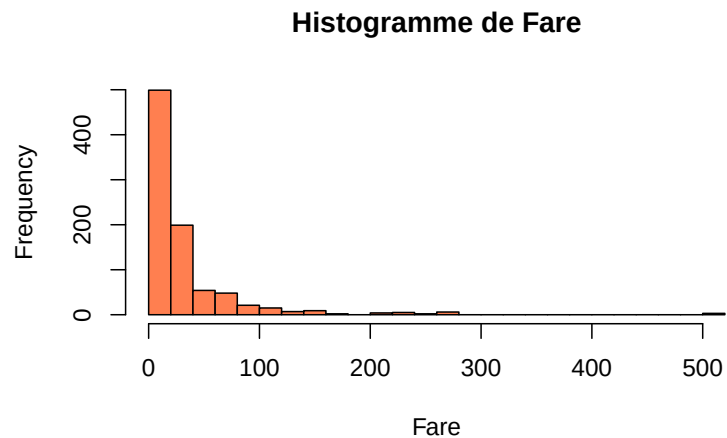


Figure 2: Distribution de Fare

Fare est très asymétrique à droite (skewed right), ce qui n'est **pas compatible** avec l'hypothèse gaussienne. On applique une transformation **logarithmique** pour se rapprocher d'une distribution normale :

```
quantitative_vars$logFare <- log(quantitative_vars$Fare + 1)

hist(quantitative_vars$logFare, breaks = 30, col = "mediumpurple",
     main = "Histogramme de log(Fare + 1)", xlab = "log(Fare + 1)")
```

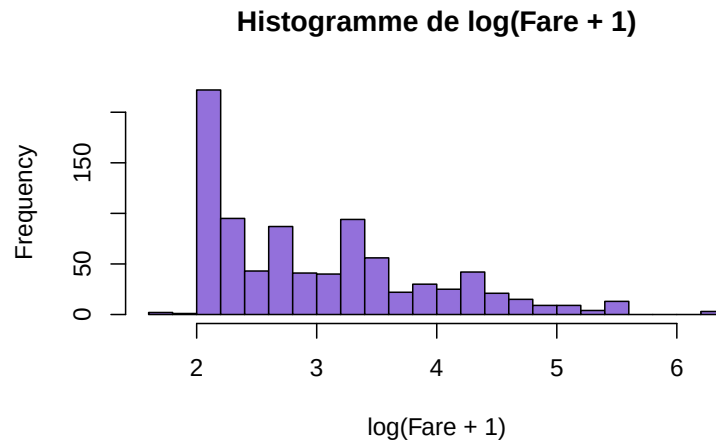


Figure 3: Distribution de $\log(\text{Fare} + 1)$

La distribution de $\log(\text{Fare} + 1)$ est bien plus proche d'une gaussienne.

2.3 Variables explicatives utilisées pour la prédiction

```
predictive_vars <- data.frame(
  qualitative_vars[, c("Pclass", "Sex", "withFam")],
  Age      = quantitative_vars$Age,
  logFare  = quantitative_vars$logFare
)
str(predictive_vars)
```

```
'data.frame':  874 obs. of  5 variables:
 $ Pclass : Factor w/ 3 levels "1","2","3": 3 1 3 1 3 3 1 3 3 2 ...
 $ Sex    : Factor w/ 3 levels "", "female", "male": 3 2 2 2 3 3 3 3 2 2 ...
 $ withFam: Factor w/ 2 levels "0","1": 2 2 1 2 1 1 1 2 2 2 ...
 $ Age    : num  22 38 26 35 35 NA 54 2 27 14 ...
 $ logFare: num  2.11 4.28 2.19 3.99 2.2 ...
```

3 Classifieur naïf bayésien avec Holdout

3.1 Partage du jeu de données

```
set.seed(1234)
n <- nrow(predictive_vars)
inTrain <- sample(1:n, size = round(0.8 * n))

predictive_vars_train <- predictive_vars[inTrain, ]
predictive_vars_test  <- predictive_vars[-inTrain, ]
target_train          <- target[inTrain]
target_test           <- target[-inTrain]
```

3.2 Gestion des données manquantes

On utilise la médiane imputation pour Age, en calculant la médiane sur le Train uniquement pour éviter le data leakage :

```
library(caret)

Proc <- preProcess(predictive_vars_train, method = "medianImpute")
predictive_vars_train <- predict(Proc, predictive_vars_train)
predictive_vars_test  <- predict(Proc, predictive_vars_test)
```

3.3 Phase d'apprentissage

```
library(naivebayes)

classif <- naive_bayes(x = predictive_vars_train, y = target_train)
```

💡 Que retourne classif ?

L'objet retourné contient les distributions conditionnelles estimées de chaque variable explicative sachant la target :

- Pour les variables **qualitatives** : les probabilités conditionnelles $\hat{P}(X^{(l)} = x|Y = y)$ (tables de probabilités).
- Pour les variables **quantitatives** : les paramètres $(\hat{\mu}, \hat{\sigma})$ de la loi normale condi-

- tionnelle estimée.
- Les **probabilités a priori** $\hat{P}(Y = y)$.

```
classif
```

```
===== Naive Bayes =====
```

Call:

```
naive_bayes.default(x = predictive_vars_train, y = target_train)
```

```
-----
```

Laplace smoothing: 0

```
-----
```

A priori probabilities:

	Décédé	Survivant
	0.6051502	0.3948498

```
-----
```

Tables:

```
-----
```

:: Pclass (Categorical)

```
-----
```

Pclass	Décédé	Survivant
1	0.1489362	0.3876812
2	0.1607565	0.2536232
3	0.6903073	0.3586957

```
-----
```

:: Sex (Categorical)

```
-----
```

Sex	Décédé	Survivant
	0.0000000	0.0000000
female	0.1442080	0.6811594
male	0.8557920	0.3188406

```
:: withFam (Bernoulli)
```

```
withFam      Décédé Survivant
      0 0.6619385 0.4782609
      1 0.3380615 0.5217391
```

```
:: Age (Gaussian)
```

```
Age          Décédé Survivant
  mean 29.74113 27.99518
  sd   12.15621 13.73591
```

```
:: logFare (Gaussian)
```

```
logFare      Décédé Survivant
  mean 2.7888601 3.3620149
  sd   0.7740001 0.9716655
```

Représentation graphique des distributions conditionnelles :

```
plot(classif, prob = "conditional")
```

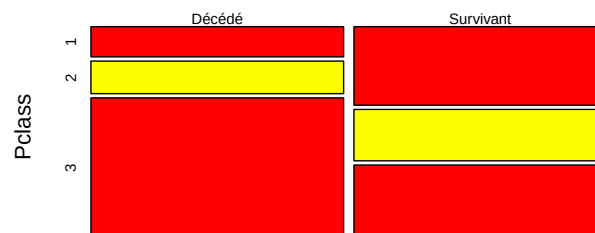


Figure 4: Distributions conditionnelles des variables explicatives

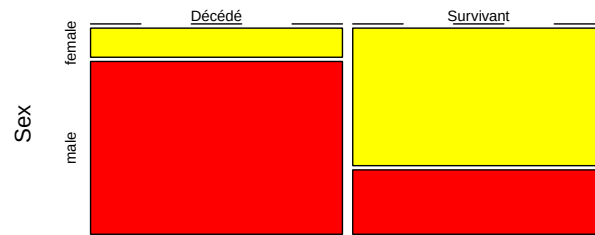


Figure 5: Distributions conditionnelles des variables explicatives

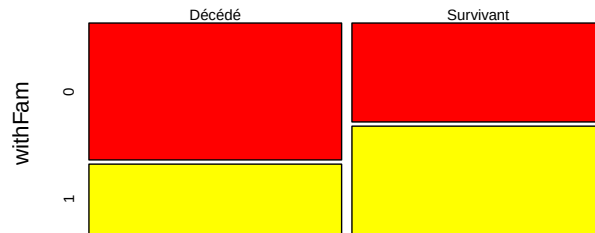


Figure 6: Distributions conditionnelles des variables explicatives

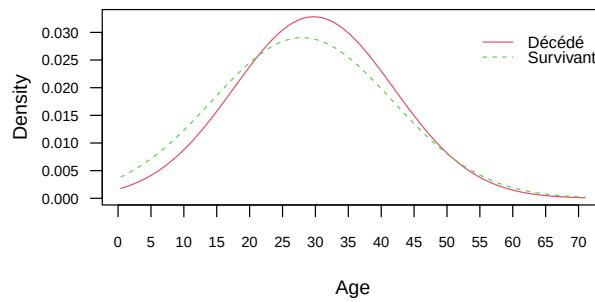


Figure 7: Distributions conditionnelles des variables explicatives

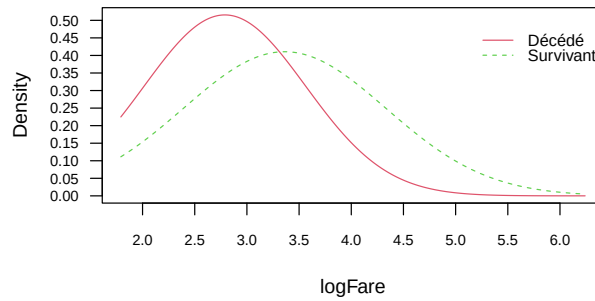


Figure 8: Distributions conditionnelles des variables explicatives

i Laplace smoothing

Si une combinaison catégorie-réponse n'apparaît pas dans les données d'entraînement, on obtient $\hat{P}(X^{(l)} = x^{(l)}|Y) = 0$. L'argument `laplace` de `naive_bayes` permet d'appliquer un lissage additif pour éviter ce problème.

3.4 Prédiction des probabilités

```
pred <- predict(classif, predictive_vars_test, type = "prob")
head(pred)
```

	Décédé	Survivant
[1,]	0.9610238	0.03897618
[2,]	0.7561169	0.24388313
[3,]	0.4372293	0.56277066
[4,]	0.2715813	0.72841872
[5,]	0.8677017	0.13229833
[6,]	0.6546453	0.34535469

Cette ligne prédit pour chaque individu du jeu de test la **probabilité d'appartenir à chaque catégorie** de la target (Décédé / Survivant).

Pour obtenir directement la **catégorie prédite**, on utilise `type = "class"` au lieu de `type = "prob"`.

3.5 Prédiction et évaluation

```
target_pred <- predict(classif, predictive_vars_test, type = "class")
cm <- confusionMatrix(target_pred, target_test, positive = "Survivant")
cm
```

Confusion Matrix and Statistics

	Reference	
Prediction	Décédé	Survivant
Décédé	95	17
Survivant	16	47

Accuracy : 0.8114
95% CI : (0.7455, 0.8665)
No Information Rate : 0.6343
P-Value [Acc > NIR] : 2.542e-07

Kappa : 0.5922

McNemar's Test P-Value : 1

Sensitivity : 0.7344
Specificity : 0.8559
Pos Pred Value : 0.7460
Neg Pred Value : 0.8482
Prevalence : 0.3657
Detection Rate : 0.2686
Detection Prevalence : 0.3600
Balanced Accuracy : 0.7951

'Positive' Class : Survivant

```
cat("Accuracy :", round(cm$overall["Accuracy"], 4), "\n")
```

Accuracy : 0.8114

```
cat("Recall :", round(cm$byClass["Sensitivity"], 4), "\n")
```

Recall : 0.7344

```
cat("Precision :", round(cm$byClass["Pos Pred Value"], 4), "\n")
```

Precision : 0.746

⚠ Sensibilité au seed

En recompilant avec d'autres seeds, les métriques varient car le partage Train/Test est aléatoire. On pourrait proposer d'utiliser la **validation croisée** pour obtenir des estimations plus stables des performances.

3.6 Pour aller plus loin : importance des variables

On peut visualiser l'influence de chaque variable explicative sur les probabilités prédites.

Pour une variable **qualitative** comme Sex :

```
boxplot(pred[, 2] ~ predictive_vars_test$Sex, col = c("pink", "lightblue"),
        main = "Importance de Sex", ylab = "P(Survivant)", xlab = "Sex")
```

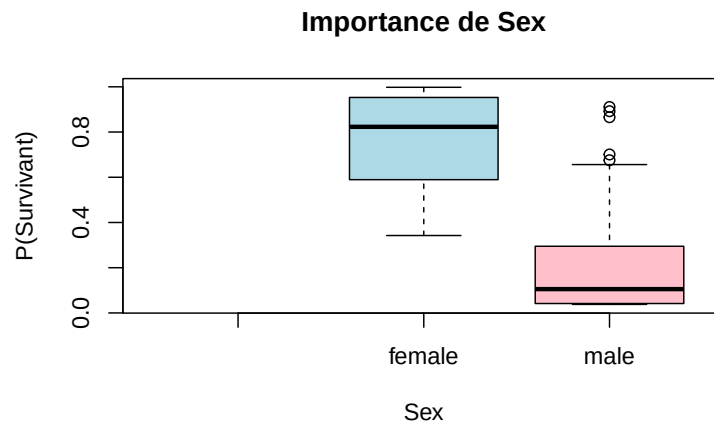


Figure 9: Probabilité prédite de survivre selon Sex

Pour une variable **quantitative** comme logFare, on utiliserait un **nuage de points** (scatter plot) :

```
plot(predictive_vars_test$logFare, pred[, 2],
     pch = 16, col = adjustcolor("steelblue", 0.5),
     xlab = "log(Fare + 1)", ylab = "P(Survivant)",
     main = "Importance de logFare")
```

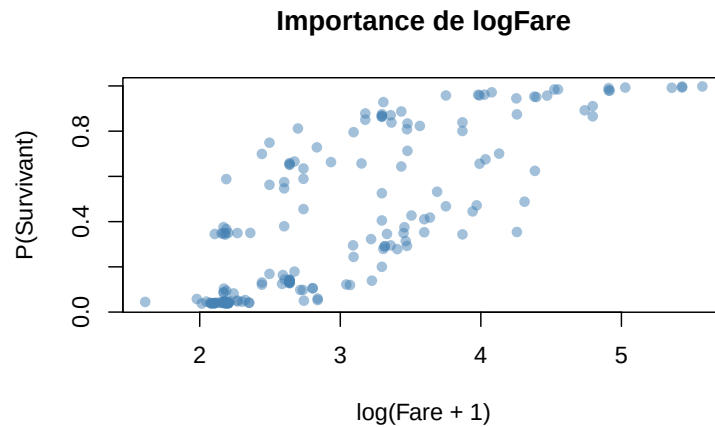


Figure 10: Probabilité prédite de survivre selon $\log(\text{Fare})$

4 Classifieur naïf bayésien avec 10-fold cross-validation

4.1 Assignment des folds

```
set.seed(1234)
nfolds <- 10
folds <- sample(1:nfolds, n, replace = TRUE)
```

4.2 Boucle de validation croisée

```
accuracy_fold <- numeric(nfolds)
recall_fold <- numeric(nfolds)
precision_fold <- numeric(nfolds)

for (k in 1:nfolds) {
  inFold <- which(folds == k)

  predictive_vars_test <- predictive_vars[inFold, ]
  target_test <- target[inFold]
  predictive_vars_train <- predictive_vars[-inFold, ]
  target_train <- target[-inFold]

  # Median imputation (sans data leakage)
  Proc <- preProcess(predictive_vars_train, method = "medianImpute")
```

```

predictive_vars_train <- predict(Proc, predictive_vars_train)
predictive_vars_test  <- predict(Proc, predictive_vars_test)

# Apprentissage et prédiction
classif      <- naive_bayes(x = predictive_vars_train, y = target_train)
target_pred <- predict(classif, predictive_vars_test, type = "class")

# Calcul des métriques
cm <- confusionMatrix(target_pred, target_test, positive = "Survivant")

accuracy_fold[k] <- cm$overall["Accuracy"]
recall_fold[k]   <- cm$byClass["Sensitivity"]
precision_fold[k] <- cm$byClass["Pos Pred Value"]
}

names(accuracy_fold) <- paste0("fold_", 1:nfolds)
names(recall_fold)   <- paste0("fold_", 1:nfolds)
names(precision_fold) <- paste0("fold_", 1:nfolds)

```

4.3 Résultats par fold

accuracy_fold

fold_1	fold_2	fold_3	fold_4	fold_5	fold_6	fold_7	fold_8
0.7555556	0.7125000	0.7500000	0.7272727	0.7674419	0.7345133	0.7241379	0.8089888
fold_9	fold_10						
0.7236842	0.8000000						

recall_fold

fold_1	fold_2	fold_3	fold_4	fold_5	fold_6	fold_7	fold_8
0.7187500	0.5416667	0.5937500	0.6571429	0.5789474	0.6296296	0.6363636	0.8437500
fold_9	fold_10						
0.5862069	0.7741935						

precision_fold

fold_1	fold_2	fold_3	fold_4	fold_5	fold_6	fold_7	fold_8
0.6388889	0.5200000	0.6333333	0.7187500	0.8461538	0.7727273	0.6363636	0.6923077
fold_9	fold_10						
0.6538462	0.7272727						

💡 Commentaire

Les valeurs varient d'un fold à l'autre car chaque fold utilise un sous-ensemble différent pour le test. C'est normal et c'est précisément ce que la CV cherche à capturer : la variabilité des performances selon le découpage.

4.4 Moyennes cross-validées

```
cat("10-fold CV Mean Accuracy :", round(mean(accuracy_fold), 4), "\n")
```

```
10-fold CV Mean Accuracy : 0.7504
```

```
cat("10-fold CV Mean Recall :", round(mean(recall_fold), 4), "\n")
```

```
10-fold CV Mean Recall : 0.656
```

```
cat("10-fold CV Mean Precision :", round(mean(precision_fold, na.rm = TRUE), 4), "\n")
```

```
10-fold CV Mean Precision : 0.684
```

i Stabilité

En recompilant avec une autre seed, les résultats moyens sont **beaucoup plus stables** qu'avec le Holdout simple. C'est l'avantage principal de la validation croisée : elle réduit la dépendance au découpage aléatoire.

5 Pour aller plus loin

5.1 Bonus : version catégorielle de Age

On crée 4 catégories : enfants (< 13), adolescents (13-18), adultes (18-60), seniors (> 60).

5.1.1 Avec Holdout

```
Age_cat <- cut(quantitative_vars$Age,
              breaks = c(-Inf, 13, 18, 60, Inf),
              labels = c("Enfant", "Adolescent", "Adulte", "Senior"))

predictive_vars_cat <- data.frame(
  qualitative_vars[, c("Pclass", "Sex", "withFam")],
  Age_cat = Age_cat,
  logFare = quantitative_vars$logFare
)

set.seed(1234)
inTrain <- sample(1:n, size = round(0.8 * n))

train_cat <- predictive_vars_cat[inTrain, ]
test_cat  <- predictive_vars_cat[-inTrain, ]
y_train   <- target[inTrain]
y_test    <- target[-inTrain]

# Imputation mode pour Age_cat, médiane pour logFare
# (pas de NA dans logFare, donc on gère juste Age_cat)
# Si Age est NA, Age_cat sera NA => on impute par le mode
mode_age_cat <- names(which.max(table(train_cat$Age_cat)))
train_cat$Age_cat[is.na(train_cat$Age_cat)] <- mode_age_cat
test_cat$Age_cat[is.na(test_cat$Age_cat)]   <- mode_age_cat

classif_cat <- naive_bayes(x = train_cat, y = y_train)
pred_cat    <- predict(classif_cat, test_cat, type = "class")
cm_cat      <- confusionMatrix(pred_cat, y_test, positive = "Survivant")

cat("Accuracy  :", round(cm_cat$overall["Accuracy"], 4), "\n")
```

Accuracy : 0.8114

```
cat("Recall    :", round(cm_cat$byClass["Sensitivity"], 4), "\n")
```

Recall : 0.7188


```
cat("Precision :", round(cm_cat$byClass["Pos Pred Value"], 4), "\n")
```

Precision : 0.7541

5.1.2 Avec 10-fold CV

```
set.seed(1234)
folds_cat <- sample(1:nfolds, n, replace = TRUE)

acc_cat <- numeric(nfolds)
rec_cat <- numeric(nfolds)
prec_cat <- numeric(nfolds)

for (k in 1:nfolds) {
  inFold <- which(folds_cat == k)

  train_k <- predictive_vars_cat[-inFold, ]
  test_k <- predictive_vars_cat[inFold, ]
  y_tr <- target[-inFold]
  y_te <- target[inFold]

  # Imputation mode pour Age_cat
  mode_val <- names(which.max(table(train_k$Age_cat)))
  train_k$Age_cat[is.na(train_k$Age_cat)] <- mode_val
  test_k$Age_cat[is.na(test_k$Age_cat)] <- mode_val

  classif_k <- naive_bayes(x = train_k, y = y_tr)
  pred_k <- predict(classif_k, test_k, type = "class")
  cm_k <- confusionMatrix(pred_k, y_te, positive = "Survivant")

  acc_cat[k] <- cm_k$overall["Accuracy"]
  rec_cat[k] <- cm_k$byClass["Sensitivity"]
  prec_cat[k] <- cm_k$byClass["Pos Pred Value"]
}

cat("10-fold CV Mean Accuracy :", round(mean(acc_cat), 4), "\n")
```

10-fold CV Mean Accuracy : 0.7574

```
cat("10-fold CV Mean Recall      :", round(mean(rec_cat), 4), "\n")
```

10-fold CV Mean Recall : 0.6639

```
cat("10-fold CV Mean Precision :", round(mean(prec_cat, na.rm = TRUE), 4), "\n")
```

10-fold CV Mean Precision : 0.6921

5.2 Bonus : version catégorielle de Fare

```
Fare_cat <- cut(quantitative_vars$Fare,
               breaks = quantile(quantitative_vars$Fare, probs = c(0, 0.25, 0.5, 0.75, 1),
                                include.lowest = TRUE),
               labels = c("Bas", "Moyen-Bas", "Moyen-Haut", "Haut"),
               include.lowest = TRUE)

predictive_vars_allcat <- data.frame(
  qualitative_vars[, c("Pclass", "Sex", "withFam")],
  Age_cat = Age_cat,
  Fare_cat = Fare_cat
)

set.seed(1234)
folds_allcat <- sample(1:nfolds, n, replace = TRUE)

acc_allcat <- numeric(nfolds)
rec_allcat <- numeric(nfolds)
prec_allcat <- numeric(nfolds)

for (k in 1:nfolds) {
  inFold <- which(folds_allcat == k)

  train_k <- predictive_vars_allcat[-inFold, ]
  test_k <- predictive_vars_allcat[inFold, ]
  y_tr <- target[-inFold]
  y_te <- target[inFold]

  # Imputation mode
  mode_age <- names(which.max(table(train_k$Age_cat)))
  mode_fare <- names(which.max(table(train_k$Fare_cat)))
```

```

train_k$Age_cat[is.na(train_k$Age_cat)] <- mode_age
test_k$Age_cat[is.na(test_k$Age_cat)] <- mode_age
train_k$Fare_cat[is.na(train_k$Fare_cat)] <- mode_fare
test_k$Fare_cat[is.na(test_k$Fare_cat)] <- mode_fare

classif_k <- naive_bayes(x = train_k, y = y_tr)
pred_k <- predict(classif_k, test_k, type = "class")
cm_k <- confusionMatrix(pred_k, y_te, positive = "Survivant")

acc_allcat[k] <- cm_k$overall["Accuracy"]
rec_allcat[k] <- cm_k$byClass["Sensitivity"]
prec_allcat[k] <- cm_k$byClass["Pos Pred Value"]
}

cat("10-fold CV Mean Accuracy :", round(mean(acc_allcat), 4), "\n")

```

10-fold CV Mean Accuracy : 0.7464

```

cat("10-fold CV Mean Recall :", round(mean(rec_allcat), 4), "\n")

```

10-fold CV Mean Recall : 0.6669

```

cat("10-fold CV Mean Precision :", round(mean(prec_allcat, na.rm = TRUE), 4), "\n")

```

10-fold CV Mean Precision : 0.6706